

REDUX

Il progetto

Un'applicazione per cercare e "acquistare" foto da **Unsplash**: ricerca, paginazione, carrello, checkout con form validato.

Stack completo: React + React Router v6 + Redux Toolkit + Axios + styled-components + styled-system + Formik + Yup.

PARTE A — IL DESIGN SYSTEM (styled-system)

Styled System — cos'è

Definizione. `styled-system` è una libreria che si appoggia a `styled-components` e permette di **passare gli stili come props**, prendendo i valori da un tema centralizzato.

```
npm install styled-components styled-system
```

Invece di scrivere CSS per ogni variazione, scrivi:

```
<Box width="100%" p={3} bg="primary" mb={4} />
```

Il Box Component — il mattone di base:

```
import styled from 'styled-components';
import { space, layout, color, flexbox, border, typography } from 'styled-system';

const Box = styled('div')(
  space,      // m, mt, mb, p, px, py...
  layout,     // width, height, display, maxWidth...
  color,      // color, bg
  flexbox,    // display:flex, alignItems, justifyContent...
  border,     // border, borderRadius...
  typography // fontSize, fontWeight, textAlign...
);

export default Box;
```

Nota la sintassi `styled('div')(...)` con le **parentesi tonde** invece di `styled.div\...\`` con i template literal. È la forma che accetta funzioni di styled-system come argomenti, ed è quella che il corso usa da qui in poi.

Il tema è dove vivono i valori reali:

```
// theme.js

const theme = {
  colors: {
    primary: '#3d5a80',
    secondary: '#ee6c4d',
    dark: '#293241',
    light: '#e0fbfc',
    grey: '#98c1d9'
  },
  space: [0, 4, 8, 16, 32, 64, 128], // scala di spaziatura
  fontSizes: [12, 14, 16, 20, 24, 32, 48],
  breakpoints: ['576px', '768px', '992px', '1200px']
};

export default theme;
```

```
// index.js

import { ThemeProvider } from 'styled-components';

<ThemeProvider theme={theme}>
  <App />
</ThemeProvider>
```

Il meccanismo da capire: `p={3}` non significa "3 pixel". Significa "**il quarto valore della scala space**", cioè 16px. `bg="primary"` prende `colors.primary`. Tutti i valori dell'interfaccia passano da un unico file, e cambiare il tema cambia l'intera applicazione.

Questo è ciò che si intende con **design system**: non stili scritti ovunque, ma un vocabolario condiviso e limitato.

Styled System — Variants

Definizione. Le *variants* sono gruppi di stili predefiniti, selezionabili con una sola prop.

```
import { variant } from 'styled-system';
```

```
const Text = styled('p')(
  space, color, typography,
  variant({
    variants: {
      title: { fontSize: 5, fontWeight: 'bold', color: 'dark' },
      subtitle: { fontSize: 3, color: 'grey' },
      body: { fontSize: 2, lineHeight: 1.6 },
      caption: { fontSize: 0, color: 'grey', fontStyle: 'italic' }
    }
  })
);

// uso
<Text variant="title">Wiki Photos</Text>
<Text variant="caption">Foto di Unsplash</Text>
```

Perché sono utili. Impediscono la proliferazione di stili quasi-uguali. Chi scrive il componente non decide arbitrariamente la dimensione del testo: sceglie fra le varianti esistenti. È il modo in cui i design system reali garantiscono coerenza.

Stack Component

Definizione. Un componente il cui unico scopo è **distribuire i figli** con una spaziatura uniforme. È uno dei pattern più utili di qualsiasi design system.

```
const Stack = ({ children, space = 3, direction = 'column', ...props }) => (
  <Box
    display="flex"
    flexDirection={direction}
    sx={{ '& > * + *': { [direction === 'column' ? 'marginTop' : 'marginLeft']:
      space } }} {...props}
  >
    {children}
  </Box>
```

```
);
```

L'idea: invece di mettere `margin-bottom` su ogni elemento (e poi dover togliere quello dell'ultimo), il contenitore gestisce la spaziatura. Il selettore `& > * + *` significa "ogni figlio che ha un fratello prima di sé" — quindi tutti tranne il primo.

Input, Skeleton, Button e Layout

Input — un campo stilizzato dal tema:

```
const Input = styled('input')(
  space, color, layout, border, typography,
  { outline: 'none', '&:focus': { borderColor: 'primary' } }
);
```

Skeleton — il segnaposto animato mostrato durante il caricamento:

```
const Skeleton = styled(Box)`
  background: linear-gradient(90deg, #f0f0f0 25%, #e0e0e0 50%, #f0f0f0 75%);
  background-size: 200% 100%;
  animation: shimmer 1.5s infinite;

  @keyframes shimmer {
    0% { background-position: 200% 0; }
    100% { background-position: -200% 0; }
  }
`;
```

Perché uno skeleton e non uno spinner. Lo skeleton mostra la *forma* del contenuto che sta arrivando, riducendo il salto visivo quando i dati caricano. È lo standard usato da Facebook, LinkedIn, YouTube.

MediaQuery — la **sintassi ad array**, la caratteristica più elegante di styled-system:

```
<Box width={[1, 1/2, 1/3]} p={[2, 3, 4]} />
```

Si legge: larghezza 100% su mobile, 50% da tablet, 33% da desktop. I valori corrispondono in ordine ai `breakpoints` del tema. Tre valori di responsive in una riga, senza scrivere una sola media query.

=====

PARTE B — DATI E API

Unsplash API

Definizione. Unsplash è un archivio di foto gratuite con un'API pubblica. Richiede una registrazione e una **Access Key**.

```
// .env
```

```
REACT_APP_UNSPASH_KEY=la_tua_chiave
```

```
const key = process.env.REACT_APP_UNSPASH_KEY;
```

Le due regole del file .env:

1. Deve stare in `.gitignore`. Una chiave API pubblicata su GitHub viene trovata dai bot in minuti.
2. Con `create-react-app` il prefisso `REACT_APP_` è obbligatorio (con Vite è `VITE_`), altrimenti la variabile viene ignorata.

Gli endpoint usati:

GET /photos → foto in evidenza, con paginazione

GET /search/photos?query=... → ricerca

Axios Instance

Definizione. Axios è una libreria per le chiamate HTTP, alternativa a `fetch`. Un'**istanza** è una copia preconfigurata con URL di base e parametri fissi.

```
npm install axios
```

```
// api.js
```

```
import axios from 'axios';
```

```
const client = axios.create({
```

```
  baseURL: 'https://api.unsplash.com',
```

```
  headers: {
```

```
    Authorization: `Client-ID $
```

```
    {process.env.REACT_APP_UNSPASH_KEY}
```

```
  }
```

```
});
```

```
export default client;
```

E l'uso diventa:

```
const { data } = await client.get('/photos', { params: { page: 1, per_page: 12 } });
```

I vantaggi rispetto a fetch:

	fetch	axios
Parsing JSON	<code>await res.json()</code> manuale	automatico in <code>data</code>
Errori HTTP (404, 500)	non lancia eccezione	lancia eccezione
Header e baseURL	ripetuti ogni volta	configurati una volta
Interceptor	no	sì

Il secondo punto è decisivo: con `fetch`, un 404 **non** entra nel `catch`. Devi controllare `response.ok` a mano. Axios lancia l'errore e il `try/catch` funziona come ti aspetti.

Gli interceptor permettono di agire su ogni richiesta o risposta:

```
client.interceptors.response.use(
  (response) => response,
  (error) => {
    if (error.response?.status === 403) {
      console.error("Limite di richieste superato");
    }
    return Promise.reject(error);
  }
);
```

Rate-Limit

Definizione. Il *rate limit* è il numero massimo di richieste consentite in un intervallo di tempo. Unsplash in modalità demo concede **50 richieste all'ora**.

Superato il limite ricevi un **403**. Il progetto lo gestisce esplicitamente perché in fase di sviluppo, con l'hot reload che ricarica continuamente, si esaurisce in fretta.

```
if (error.response?.status === 403) {
  dispatch(catchError({ message: "Limite orario raggiunto. Riprova più tardi." }));
}
```

La lezione di metodo: gli errori delle API non sono tutti uguali. Un 403 da rate limit, un 401 da chiave sbagliata e un 500 dal server richiedono messaggi diversi. Trattarli tutti come "errore generico" è una scorciatoia che si paga in supporto.